

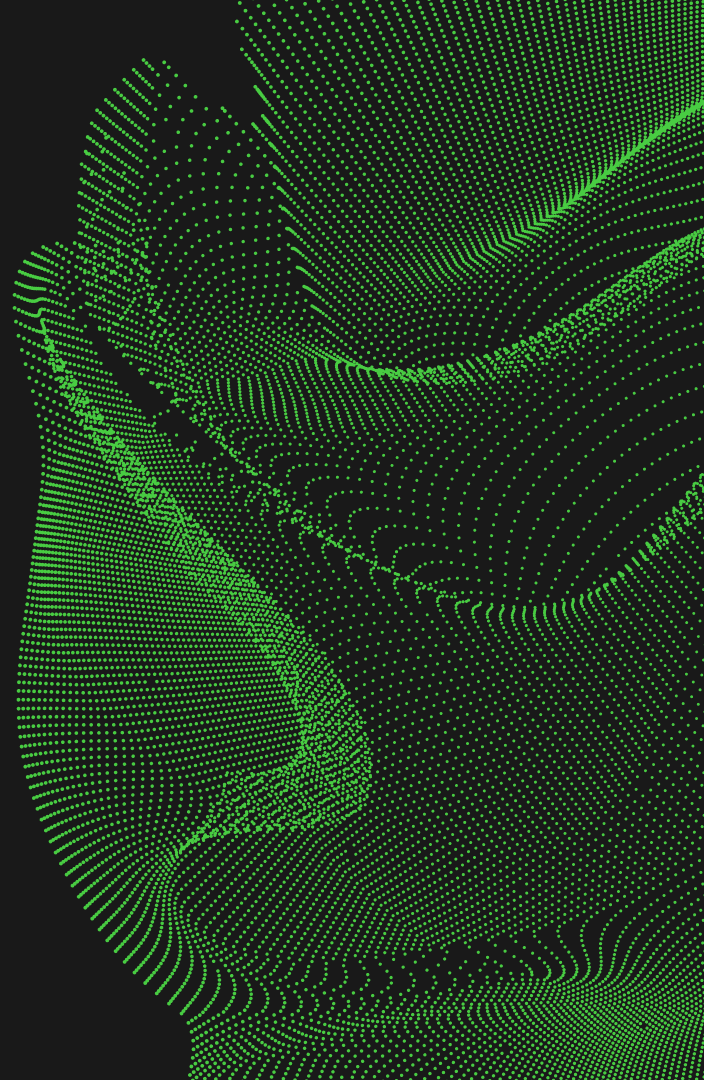


Иннополис

17-18 апреля

Профессиональная межрегиональная

IT-КОНФЕРЕНЦИЯ MERGE



Пять принципов FP в пяти языках

ДРЕВНЯЯ МАГИЯ НА КАЖДЫЙ ДЕНЬ



evrone.ru

FP guru

Павел Аргентов

*Значительная часть недостатков в разработке ПО
связана с тем, что программисты не полностью
понимают все возможные состояния, в которых может
выполняться их код*

Джон Кармак



Иннополис

17-18 апреля

ПАВЕЛ АРГЕНТОВ

- **МНОГО** опыта
- разрабатывал **всё**
- FP с **2002** г.

Evrone.ru: разработка разработчиков



$(\lambda x.M)N \rightarrow_{\beta} M[x := N]$

ФП: пять конкретных свойств кода

Смысл: описание результата, а не
микроменеджмент процесса.

```
[1,2,3].map(x ⇒ x * 2)  
const add = x ⇒ y ⇒ x + y  
const x = 1; x = 2 // TypeError
```



Иннополис

17-18 апреля

ПЯТЬ ЯЗЫКОВ

OCaml

язык богов (тм)

Scala

почти OCaml

Python

язык МЧС

JS

вездесущ

Rust

всемогущ

ДЕКЛАРАТИВНОСТЬ

- Императивный код — **микроменеджмент**
- Декларативный код — **спецификация**
- Людям — **понятность**
- Машинам — **законы оптимизации**

```
SELECT ... WHERE vs for ... in  
for ... end vs list.map
```

ДЕКЛАРАТИВНОСТЬ

Задача: сумма скидок для заказов дороже 100, скидка 10%

OCaml

```
let discount orders =  
  orders  
  ▷ List.filter (fun o → o.price > 100)  
  ▷ List.map (fun o → o.price *. 0.1)  
  ▷ List.fold_left ( +. ) 0.0
```

Scala

```
def discount(xs: List[Order]) =  
  xs  
  .filter(_.price > 100)  
  .map(_.price * 0.1)  
  .sum
```

Rust

```
fn discount(xs: Vec[Order]) → f64 {  
  xs.iter()  
  .filter(|o| o.price > 100.)  
  .map(|o| o.price * 0.1)  
  .sum()  
}
```

Python

```
def discount(orders):  
    return sum(  
        o.price * 0.1  
        for o in orders  
        if o.price > 100.0  
    )
```

JS

```
const discount = orders =>  
  orders  
  .filter(o => o.price > 100)  
  .map(o => o.price * 0.1)  
  .reduce((s,d) => s+d, 0)
```

ДЕКЛАРАТИВНОСТЬ

Задача: сумма скидок для заказов дороже 100, скидка 10%

OCaml

```
let discount orders =  
  orders  
  ▷ List.filter (fun o → o.price > 100)  
  ▷ List.map (fun o → o.price *. 0.1)  
  ▷ List.fold_left ( +. ) 0.0
```

Scala

```
def discount(xs: List[Order]) =  
  xs  
  .filter(_.price > 100)  
  .map(_.price * 0.1)  
  .sum
```

Rust

```
fn discount(xs: Vec[Order]) → f64 {  
  xs.iter()  
  .filter(|o| o.price > 100.)  
  .map(|o| o.price * 0.1)  
  .sum()  
}
```

Python

```
def discount(orders):  
    return sum(  
        o.price * 0.1  
        for o in orders  
        if o.price > 100.0  
    )
```

JS

```
const discount = orders =>  
  orders  
  .filter(o => o.price > 100)  
  .map(o => o.price * 0.1)  
  .reduce((s,d) => s+d, 0)
```


ДЕКЛАРАТИВНОСТЬ

Задача: сумма скидок для заказов дороже 100, скидка 10%

OCaml

```
let discount orders =  
  orders  
  ▷ List.filter (fun o → o.price > 100)  
  ▷ List.map (fun o → o.price *. 0.1)  
  ▷ List.fold_left ( +. ) 0.0
```

Scala

```
def discount(xs: List[Order]) =  
  xs  
  .filter(_.price > 100)  
  .map(_.price * 0.1)  
  .sum
```

Rust

```
fn discount(xs: Vec[Order]) → f64 {  
  xs.iter()  
    .filter(|o| o.price > 100.)  
    .map(|o| o.price * 0.1)  
    .sum()  
}
```

Python

```
def discount(orders):  
    return sum(  
        o.price * 0.1  
        for o in orders  
        if o.price > 100.0  
    )
```

JS

```
const discount = orders =>  
  orders  
    .filter(o => o.price > 100)  
    .map(o => o.price * 0.1)  
    .reduce((s,d) => s+d, 0)
```

ДЕКЛАРАТИВНОСТЬ

Задача: сумма скидок для заказов дороже 100, скидка 10%

OCaml

```
let discount orders =  
  orders  
  ▷ List.filter (fun o → o.price > 100)  
  ▷ List.map (fun o → o.price *. 0.1)  
  ▷ List.fold_left ( +. ) 0.0
```

Scala

```
def discount(xs: List[Order]) =  
  xs  
  .filter(_.price > 100)  
  .map(_.price * 0.1)  
  .sum
```

Rust

```
fn discount(xs: &[amp;Order]) → f64 {  
  xs.iter()  
  .filter(|o| o.price > 100.)  
  .map(|o| o.price * 0.1)  
  .sum()  
}
```

Python

```
def discount(orders):  
    return sum(  
        o.price * 0.1  
        for o in orders  
        if o.price > 100.0  
    )
```

JS

```
const discount = orders =>  
  orders  
  .filter(o => o.price > 100)  
  .map(o => o.price * 0.1)  
  .reduce((s,d) => s+d, 0)
```

ДЕКЛАРАТИВНОСТЬ

Задача: сумма скидок для заказов дороже 100, скидка 10%

OCaml

```
let discount orders =  
  orders  
  ▷ List.filter (fun o → o.price > 100)  
  ▷ List.map (fun o → o.price *. 0.1)  
  ▷ List.fold_left ( +. ) 0.0
```

Scala

```
def discount(xs: List[Order]) =  
  xs  
  .filter(_.price > 100)  
  .map(_.price * 0.1)  
  .sum
```

Rust

```
fn discount(xs: &[amp;Order]) → f64 {  
  xs.iter()  
  .filter(|o| o.price > 100.)  
  .map(|o| o.price * 0.1)  
  .sum()  
}
```

Python

```
def discount(orders):  
    return sum(  
        o.price * 0.1  
        for o in orders  
        if o.price > 100.0  
    )
```

JS

```
const discount = orders =>  
  orders  
  .filter(o => o.price > 100)  
  .map(o => o.price * 0.1)  
  .reduce((s,d) => s+d, 0)
```

ДЕКЛАРАТИВНОСТЬ

Задача: сумма скидок для заказов дороже 100, скидка 10%

OCaml

```
let discount orders =  
  orders  
  ▷ List.filter (fun o → o.price > 100)  
  ▷ List.map (fun o → o.price *. 0.1)  
  ▷ List.fold_left ( +. ) 0.0
```

Scala

```
def discount(xs: List[Order]) =  
  xs  
  .filter(_.price > 100)  
  .map(_.price * 0.1)  
  .sum
```

Rust

```
fn discount(xs: &[amp;Order]) → f64 {  
  xs.iter()  
  .filter(|o| o.price > 100.)  
  .map(|o| o.price * 0.1)  
  .sum()  
}
```

Python

```
def discount(orders):  
    return sum(  
        o.price * 0.1  
        for o in orders  
        if o.price > 100.0  
    )
```

JS

```
const discount = orders ⇒  
  orders  
  .filter(o ⇒ o.price > 100)  
  .map(o ⇒ o.price * 0.1)  
  .reduce((s,d) ⇒ s+d, 0)
```



Иннополис

17-18 апреля

ВЫРАЖЕНИЯ ВМЕСТО ИНСТРУКЦИЙ

- Инструкции производят **эффект**
- Выражения вычисляются в **значения**
- Значения можно **компоновать**



Иннополис

17-18 апреля

ВЫРАЖЕНИЯ ВМЕСТО ИНСТРУКЦИЙ

Задача: вычислить сумму чётных чисел

OCaml

```
let sum_even xs =  
  xs  
  ▷ List.filter (fun x → x mod 2 = 0)  
  ▷ List.fold_left ( + ) 0
```

Scala

```
def sumEven(xs: List[Int]) =  
  xs  
  .filter(_ % 2 == 0)  
  .sum
```

Rust

```
fn sum_even(xs: &[i32]) -> i32 {  
  xs.iter()  
    .filter(|x| *x % 2 == 0)  
    .sum()  
}
```

Python

```
def sum_even(xs):  
    return sum(  
        x for x in xs if x % 2 == 0  
    )
```

JS

```
const sumEven = xs =>  
  xs  
  .filter(x => x % 2 === 0)  
  .reduce((s, x) => s + x, 0)
```

ВЫРАЖЕНИЯ ВМЕСТО ИНСТРУКЦИЙ

Задача: вычислить сумму чётных чисел

OCaml

```
let sum_even xs =  
  xs  
  ▷ List.filter (fun x → x mod 2 = 0)  
  ▷ List.fold_left ( + ) 0
```

Scala

```
def sumEven(xs: List[Int]) =  
  xs  
  .filter(_ % 2 == 0)  
  .sum
```

Rust

```
fn sum_even(xs: &[i32]) -> i32 {  
  xs.iter()  
    .filter(|x| *x % 2 == 0)  
    .sum()  
}
```

Python

```
def sum_even(xs):  
    return sum(  
        x for x in xs if x % 2 == 0  
    )
```

JS

```
const sumEven = xs =>  
  xs  
  .filter(x => x % 2 === 0)  
  .reduce((s, x) => s + x, 0)
```

ВЫРАЖЕНИЯ ВМЕСТО ИНСТРУКЦИЙ

Задача: вычислить сумму чётных чисел

OCaml

```
let sum_even xs =  
  xs  
  ▷ List.filter (fun x → x mod 2 = 0)  
  ▷ List.fold_left ( + ) 0
```

Scala

```
def sumEven(xs: List[Int]) =  
  xs  
  .filter(_ % 2 == 0)  
  .sum
```

Rust

```
fn sum_even(xs: &[i32]) -> i32 {  
  xs.iter()  
    .filter(|x| *x % 2 == 0)  
    .sum()  
}
```

Python

```
def sum_even(xs):  
    return sum(  
        x for x in xs if x % 2 == 0  
    )
```

JS

```
const sumEven = xs =>  
  xs  
    .filter(x => x % 2 === 0)  
    .reduce((s, x) => s + x, 0)
```




Иннополис

17-18 апреля

ВЫРАЖЕНИЯ ВМЕСТО ИНСТРУКЦИЙ

Задача: вычислить сумму чётных чисел

OCaml

```
let sum_even xs =  
  xs  
  ▷ List.filter (fun x → x mod 2 = 0)  
  ▷ List.fold_left ( + ) 0
```

Scala

```
def sumEven(xs: List[Int]) =  
  xs  
  .filter(_ % 2 == 0)  
  .sum
```

Rust

```
fn sum_even(xs: &[i32]) → i32 {  
  xs.iter()  
    .filter(|x| *x % 2 == 0)  
    .sum()  
}
```

Python

```
def sum_even(xs):  
    return sum(  
        x for x in xs if x % 2 == 0  
    )
```

JS

```
const sumEven = xs ⇒  
  xs  
    .filter(x ⇒ x % 2 === 0)  
    .reduce((s, x) ⇒ s + x, 0)
```

ВЫРАЖЕНИЯ ВМЕСТО ИНСТРУКЦИЙ

Задача: вычислить сумму чётных чисел

OCaml

```
let sum_even xs =  
  xs  
  ▷ List.filter (fun x → x mod 2 = 0)  
  ▷ List.fold_left ( + ) 0
```

Scala

```
def sumEven(xs: List[Int]) =  
  xs  
  .filter(_ % 2 == 0)  
  .sum
```

Rust

```
fn sum_even(xs: &[i32]) → i32 {  
  xs.iter()  
    .filter(|x| *x % 2 == 0)  
    .sum()  
}
```

Python

```
def sum_even(xs):  
    return sum(  
        x for x in xs if x % 2 == 0  
    )
```

JS

```
const sumEven = xs ⇒  
  xs  
    .filter(x ⇒ x % 2 === 0)  
    .reduce((s, x) ⇒ s + x, 0)
```

ВЫРАЖЕНИЯ ВМЕСТО ИНСТРУКЦИЙ

Задача: вычислить сумму чётных чисел

OCaml

```
let sum_even xs =  
  xs  
  ▷ List.filter (fun x → x mod 2 = 0)  
  ▷ List.fold_left ( + ) 0
```

Scala

```
def sumEven(xs: List[Int]) =  
  xs  
  .filter(_ % 2 == 0)  
  .sum
```

Rust

```
fn sum_even(xs: &[i32]) → i32 {  
  xs.iter()  
    .filter(|x| *x % 2 == 0)  
    .sum()  
}
```

Python

```
def sum_even(xs):  
    return sum(  
        x for x in xs if x % 2 == 0  
    )
```

JS

```
const sumEven = xs ⇒  
  xs  
    .filter(x ⇒ x % 2 === 0)  
    .reduce((s, x) ⇒ s + x, 0)
```



Иннополис

17-18 апреля

ФУНКЦИИ КАК ЗНАЧЕНИЯ

- Функция — **first-class value**: передаётся аргументом, связывается с переменной, возвращается из функции
- HOF — принимает или возвращает функцию: **map, filter, fold**
- Замыкание — захват лексического окружения
- Каррирование — **$f(a, b) \rightarrow f(a)(b)$**



Иннополис

17-18 апреля

ФУНКЦИИ КАК ЗНАЧЕНИЯ

Задача: создать умножитель с коэффициентом и применить к списку

OCaml

```
let multiply a b = a * b
let triple = multiply 3
let result =
  List.map triple [1; 2; 3; 4; 5]
```

Scala

```
val multiply: (Int, Int) => Int =
  (factor, x) => x * factor

val triple: Int => Int =
  multiply.curried(3)

val result =
  List(1, 2, 3, 4, 5).map(triple)
```

Rust

```
use auto_curry::curry;
#[curry]

fn multiply(factor: i32, x: i32) -> i32 {
  factor * x
}

let triple: impl Fn(i32) -> i32 = multiply(3);
let result: Vec<i32> = vec![1, 2, 3, 4, 5]
  .into_iter().map(triple).collect();
```

Python

```
from functools import partial

def multiply(factor, x): return factor * x
triple = partial(multiply, 3)
result = list(map(triple, [1, 2, 3, 4, 5]))
```

JS

```
import * as R from "ramda";

const multiply = R.curry((factor, x) => factor * x);
const triple = multiply(3);
const result = [1, 2, 3, 4, 5].map(triple);
```



Иннополис

17-18 апреля

ФУНКЦИИ КАК ЗНАЧЕНИЯ

Задача: создать умножитель с коэффициентом и применить к списку

OCaml

```
let multiply a b = a * b
let triple = multiply 3
let result =
  List.map triple [1; 2; 3; 4; 5]
```

Scala

```
val multiply: (Int, Int) => Int =
  (factor, x) => x * factor

val triple: Int => Int =
  multiply.curried(3)

val result =
  List(1, 2, 3, 4, 5).map(triple)
```

Rust

```
use auto_curry::curry;
#[curry]

fn multiply(factor: i32, x: i32) -> i32 {
  factor * x
}

let triple: impl Fn(i32) -> i32 = multiply(3);
let result: Vec<i32> = vec![1, 2, 3, 4, 5]
  .into_iter().map(triple).collect();
```

Python

```
from functools import partial

def multiply(factor, x): return factor * x
triple = partial(multiply, 3)
result = list(map(triple, [1, 2, 3, 4, 5]))
```

JS

```
import * as R from "ramda";

const multiply = R.curry((factor, x) => factor * x);
const triple = multiply(3);
const result = [1, 2, 3, 4, 5].map(triple);
```



Иннополис

17-18 апреля

ФУНКЦИИ КАК ЗНАЧЕНИЯ

Задача: создать умножитель с коэффициентом и применить к списку

OCaml

```
let multiply a b = a * b
let triple = multiply 3
let result =
  List.map triple [1; 2; 3; 4; 5]
```

Scala

```
val multiply: (Int, Int) => Int =
  (factor, x) => x * factor

val triple: Int => Int =
  multiply.curried(3)

val result =
  List(1, 2, 3, 4, 5).map(triple)
```

Rust

```
use auto_curry::curry;
#[curry]

fn multiply(factor: i32, x: i32) -> i32 {
  factor * x
}

let triple: impl Fn(i32) -> i32 = multiply(3);
let result: Vec<i32> = vec![1, 2, 3, 4, 5]
  .into_iter().map(triple).collect();
```

Python

```
from functools import partial

def multiply(factor, x): return factor * x
triple = partial(multiply, 3)
result = list(map(triple, [1, 2, 3, 4, 5]))
```

JS

```
import * as R from "ramda";

const multiply = R.curry((factor, x) => factor * x);
const triple = multiply(3);
const result = [1, 2, 3, 4, 5].map(triple);
```

ФУНКЦИИ КАК ЗНАЧЕНИЯ

Задача: создать умножитель с коэффициентом и применить к списку

OCaml

```
let multiply a b = a * b
let triple = multiply 3
let result =
  List.map triple [1; 2; 3; 4; 5]
```

Scala

```
val multiply: (Int, Int) => Int =
  (factor, x) => x * factor

val triple: Int => Int =
  multiply.curried(3)

val result =
  List(1, 2, 3, 4, 5).map(triple)
```

Rust

```
use auto_curry::curry;
#[curry]

fn multiply(factor: i32, x: i32) -> i32 {
  factor * x
}

let triple: impl Fn(i32) -> i32 = multiply(3);
let result: Vec<i32> = vec![1, 2, 3, 4, 5]
  .into_iter().map(triple).collect();
```

Python

```
from functools import partial

def multiply(factor, x): return factor * x
triple = partial(multiply, 3)
result = list(map(triple, [1, 2, 3, 4, 5]))
```

JS

```
import * as R from "ramda";

const multiply = R.curry((factor, x) => factor * x);
const triple = multiply(3);
const result = [1, 2, 3, 4, 5].map(triple);
```




Иннополис

17-18 апреля

ФУНКЦИИ КАК ЗНАЧЕНИЯ

Задача: создать умножитель с коэффициентом и применить к списку

OCaml

```
let multiply a b = a * b
let triple = multiply 3
let result =
  List.map triple [1; 2; 3; 4; 5]
```

Scala

```
val multiply: (Int, Int) => Int =
  (factor, x) => x * factor

val triple: Int => Int =
  multiply.curried(3)

val result =
  List(1, 2, 3, 4, 5).map(triple)
```

Rust

```
use auto_curry::curry;
#[curry]

fn multiply(factor: i32, x: i32) -> i32 {
  factor * x
}

let triple: impl Fn(i32) -> i32 = multiply(3);
let result: Vec<i32> = vec![1, 2, 3, 4, 5]
  .into_iter().map(triple).collect();
```

Python

```
from functools import partial

def multiply(factor, x): return factor * x
triple = partial(multiply, 3)
result = list(map(triple, [1, 2, 3, 4, 5]))
```

JS

```
import * as R from "ramda";

const multiply = R.curry((factor, x) => factor * x);
const triple = multiply(3);
const result = [1, 2, 3, 4, 5].map(triple);
```

ФУНКЦИИ КАК ЗНАЧЕНИЯ

Задача: создать умножитель с коэффициентом и применить к списку

OCaml

```
let multiply a b = a * b
let triple = multiply 3
let result =
  List.map triple [1; 2; 3; 4; 5]
```

Scala

```
val multiply: (Int, Int) => Int =
  (factor, x) => x * factor

val triple: Int => Int =
  multiply.curried(3)

val result =
  List(1, 2, 3, 4, 5).map(triple)
```

Rust

```
use auto_curry::curry;
#[curry]

fn multiply(factor: i32, x: i32) -> i32 {
  factor * x
}

let triple: impl Fn(i32) -> i32 = multiply(3);
let result: Vec<i32> = vec![1, 2, 3, 4, 5]
  .into_iter().map(triple).collect();
```

Python

```
from functools import partial

def multiply(factor, x): return factor * x
triple = partial(multiply, 3)
result = list(map(triple, [1, 2, 3, 4, 5]))
```

JS

```
import * as R from "ramda";

const multiply = R.curry((factor, x) => factor * x);
const triple = multiply(3);
const result = [1, 2, 3, 4, 5].map(triple);
```



Иннополис

17-18 апреля

ЧИСТОТА ФУНКЦИЙ

- **Чистая функция:** возвращаемое значение зависит только от аргументов; **нет побочных эффектов.**
- Следствия: мемоизация; **независимость порядка вычислений;** data race-free **параллелизм.**
- **Эффекты** неизбежны (I/O, время, состояние). Стратегия — **изоляция** на границах системы.



Иннополис

17-18 апреля

ЧИСТОТА ФУНКЦИЙ

Задача: функция скидки — чистая и нечистая версии

OCaml

```
let discount rate price =  
  price *. (1.0 -. rate)  
  
let rate = ref 0.1  
let discount_impure price =  
  rate := !rate +. 0.05;  
  price *. (1.0 -. !rate)
```

Scala

```
def discount(rate: Double, price: Double): Double =  
  price * (1.0 - rate)  
  
var rate = 0.1  
def discountImpure(price: Double): Double = {  
  rate += 0.05; price * (1.0 - rate)  
}
```

Rust

```
fn discount(rate: f64, price: f64) -> f64 {  
  price * (1.0 - rate)  
}  
  
static mut RATE: f64 = 0.1;  
fn discount_impure(price: f64) -> f64 {  
  unsafe {  
    RATE += 0.05;  
    price * (1.0 - RATE)  
  }  
}
```

Python

```
def discount(rate, price):  
    return price * (1 - rate)  
  
rate = 0.1  
def discount_impure(price):  
    global rate  
    rate += 0.05  
    return price * (1 - rate)
```

JS

```
const discount = (rate, price) =>  
  price * (1 - rate);  
  
let rate = 0.1;  
function discountImpure(price) {  
  rate += 0.05;  
  return price * (1 - rate);  
}
```



Иннополис

17-18 апреля

ЧИСТОТА ФУНКЦИЙ

Задача: функция скидки — чистая и нечистая версии

OCaml

```
let discount rate price =  
  price *. (1.0 -. rate)  
  
let rate = ref 0.1  
let discount_impure price =  
  rate := !rate +. 0.05;  
  price *. (1.0 -. !rate)
```

Scala

```
def discount(rate: Double, price: Double): Double =  
  price * (1.0 - rate)  
  
var rate = 0.1  
def discountImpure(price: Double): Double = {  
  rate += 0.05; price * (1.0 - rate)  
}
```

Rust

```
fn discount(rate: f64, price: f64) -> f64 {  
  price * (1.0 - rate)  
}  
  
static mut RATE: f64 = 0.1;  
fn discount_impure(price: f64) -> f64 {  
  unsafe {  
    RATE += 0.05;  
    price * (1.0 - RATE)  
  }  
}
```

Python

```
def discount(rate, price):  
    return price * (1 - rate)  
  
rate = 0.1  
def discount_impure(price):  
    global rate  
    rate += 0.05  
    return price * (1 - rate)
```

JS

```
const discount = (rate, price) =>  
  price * (1 - rate);  
  
let rate = 0.1;  
function discountImpure(price) {  
  rate += 0.05;  
  return price * (1 - rate);  
}
```



Иннополис

17-18 апреля

ЧИСТОТА ФУНКЦИЙ

Задача: функция скидки — чистая и нечистая версии

OCaml

```
let discount rate price =  
  price *. (1.0 -. rate)  
  
let rate = ref 0.1  
let discount_impure price =  
  rate := !rate +. 0.05;  
  price *. (1.0 -. !rate)
```

Scala

```
def discount(rate: Double, price: Double): Double =  
  price * (1.0 - rate)  
  
var rate = 0.1  
def discountImpure(price: Double): Double = {  
  rate += 0.05; price * (1.0 - rate)  
}
```

Rust

```
fn discount(rate: f64, price: f64) -> f64 {  
  price * (1.0 - rate)  
}  
  
static mut RATE: f64 = 0.1;  
fn discount_impure(price: f64) -> f64 {  
  unsafe {  
    RATE += 0.05;  
    price * (1.0 - RATE)  
  }  
}
```

Python

```
def discount(rate, price):  
    return price * (1 - rate)  
  
rate = 0.1  
def discount_impure(price):  
    global rate  
    rate += 0.05  
    return price * (1 - rate)
```

JS

```
const discount = (rate, price) =>  
  price * (1 - rate);  
  
let rate = 0.1;  
function discountImpure(price) {  
  rate += 0.05;  
  return price * (1 - rate);  
}
```



Иннополис

17-18 апреля

ЧИСТОТА ФУНКЦИЙ

Задача: функция скидки — чистая и нечистая версии

OCaml

```
let discount rate price =  
  price *. (1.0 -. rate)  
  
let rate = ref 0.1  
let discount_impure price =  
  rate := !rate +. 0.05;  
  price *. (1.0 -. !rate)
```

Scala

```
def discount(rate: Double, price: Double): Double =  
  price * (1.0 - rate)  
  
var rate = 0.1  
def discountImpure(price: Double): Double = {  
  rate += 0.05; price * (1.0 - rate)  
}
```

Rust

```
fn discount(rate: f64, price: f64) → f64 {  
  price * (1.0 - rate)  
}  
  
static mut RATE: f64 = 0.1;  
fn discount_impure(price: f64) → f64 {  
  unsafe {  
    RATE += 0.05;  
    price * (1.0 - RATE)  
  }  
}
```

Python

```
def discount(rate, price):  
    return price * (1 - rate)  
  
rate = 0.1  
def discount_impure(price):  
    global rate  
    rate += 0.05  
    return price * (1 - rate)
```

JS

```
const discount = (rate, price) =>  
  price * (1 - rate);  
  
let rate = 0.1;  
function discountImpure(price) {  
  rate += 0.05;  
  return price * (1 - rate);  
}
```



Иннополис

17-18 апреля

ЧИСТОТА ФУНКЦИЙ

Задача: функция скидки — чистая и нечистая версии

OCaml

```
let discount rate price =  
  price *. (1.0 -. rate)  
  
let rate = ref 0.1  
let discount_impure price =  
  rate := !rate +. 0.05;  
  price *. (1.0 -. !rate)
```

Scala

```
def discount(rate: Double, price: Double): Double =  
  price * (1.0 - rate)  
  
var rate = 0.1  
def discountImpure(price: Double): Double = {  
  rate += 0.05; price * (1.0 - rate)  
}
```

Rust

```
fn discount(rate: f64, price: f64) → f64 {  
  price * (1.0 - rate)  
}  
  
static mut RATE: f64 = 0.1;  
fn discount_impure(price: f64) → f64 {  
  unsafe {  
    RATE += 0.05;  
    price * (1.0 - RATE)  
  }  
}
```

Python

```
def discount(rate, price):  
    return price * (1 - rate)  
  
rate = 0.1  
def discount_impure(price):  
    global rate  
    rate += 0.05  
    return price * (1 - rate)
```

JS

```
const discount = (rate, price) =>  
  price * (1 - rate);  
  
let rate = 0.1;  
function discountImpure(price) {  
  rate += 0.05;  
  return price * (1 - rate);  
}
```




Иннополис

17-18 апреля

ЧИСТОТА ФУНКЦИЙ

Задача: функция скидки — чистая и нечистая версии

OCaml

```
let discount rate price =  
  price *. (1.0 -. rate)  
  
let rate = ref 0.1  
let discount_impure price =  
  rate := !rate +. 0.05;  
  price *. (1.0 -. !rate)
```

Scala

```
def discount(rate: Double, price: Double): Double =  
  price * (1.0 - rate)  
  
var rate = 0.1  
def discountImpure(price: Double): Double = {  
  rate += 0.05; price * (1.0 - rate)  
}
```

Rust

```
fn discount(rate: f64, price: f64) → f64 {  
  price * (1.0 - rate)  
}  
  
static mut RATE: f64 = 0.1;  
fn discount_impure(price: f64) → f64 {  
  unsafe {  
    RATE += 0.05;  
    price * (1.0 - RATE)  
  }  
}
```

Python

```
def discount(rate, price):  
    return price * (1 - rate)  
  
rate = 0.1  
def discount_impure(price):  
    global rate  
    rate += 0.05  
    return price * (1 - rate)
```

JS

```
const discount = (rate, price) =>  
  price * (1 - rate);  
  
let rate = 0.1;  
function discountImpure(price) {  
  rate += 0.05;  
  return price * (1 - rate);  
}
```



Иннополис

17-18 апреля

ИММУТАБЕЛЬНОСТЬ

- `const` / `val` — запрет перезаписи переменной. Иммутабельность — **запрет мутации самого значения.**
- Гарантия: **компилятором** (OCaml `let`, Rust `let`) / **соглашением** (Scala `val`) / **опционально** (Python `frozen=True`, JS `Object.freeze`).
- Persistent data structures: **structural sharing**. Обновление поля — **$O(\log n)$, не $O(n)$.**



Иннополис

17-18 апреля

ИММУТАБЕЛЬНОСТЬ

Задача: обновить одно поле записи без изменения оригинала

OCaml

```
type user = { name: string; age: int }  
  
let user = { name = "Alice"; age = 30 }  
let older = { user with age = 31 }
```

Scala

```
case class User(name: String, age: Int)  
  
val user = User("Alice", 30)  
val older = user.copy(age = 31)
```

Rust

```
struct User { name: String, age: u32 }  
  
let user = User { name: "Alice".into(), age: 30 };  
let older = User { age: 31, ..user };  
  
let mut draft = User { name: "Bob".into(), age: 25 };  
draft.age = 26;
```

Python

```
from dataclasses import dataclass, replace  
  
@dataclass(frozen=True)  
class User: name: str; age: int  
  
user = User("Alice", 30)  
older = replace(user, age=31)
```

JS

```
"use strict";  
  
const user = Object.freeze({ name: "Alice", age: 30 });  
const older = { ...user, age: 31 };
```



Иннополис

17-18 апреля

ИММУТАБЕЛЬНОСТЬ

Задача: обновить одно поле записи без изменения оригинала

OCaml

```
type user = { name: string; age: int }  
  
let user = { name = "Alice"; age = 30 }  
let older = { user with age = 31 }
```

Scala

```
case class User(name: String, age: Int)  
  
val user = User("Alice", 30)  
val older = user.copy(age = 31)
```

Rust

```
struct User { name: String, age: u32 }  
  
let user = User { name: "Alice".into(), age: 30 };  
let older = User { age: 31, ..user };  
  
let mut draft = User { name: "Bob".into(), age: 25 };  
draft.age = 26;
```

Python

```
from dataclasses import dataclass, replace  
  
@dataclass(frozen=True)  
class User: name: str; age: int  
  
user = User("Alice", 30)  
older = replace(user, age=31)
```

JS

```
"use strict";  
  
const user = Object.freeze({ name: "Alice", age: 30 });  
const older = { ...user, age: 31 };
```



Иннополис

17-18 апреля

ИММУТАБЕЛЬНОСТЬ

Задача: обновить одно поле записи без изменения оригинала

OCaml

```
type user = { name: string; age: int }  
  
let user = { name = "Alice"; age = 30 }  
let older = { user with age = 31 }
```

Scala

```
case class User(name: String, age: Int)  
  
val user = User("Alice", 30)  
val older = user.copy(age = 31)
```

Rust

```
struct User { name: String, age: u32 }  
  
let user = User { name: "Alice".into(), age: 30 };  
let older = User { age: 31, ..user };  
  
let mut draft = User { name: "Bob".into(), age: 25 };  
draft.age = 26;
```

Python

```
from dataclasses import dataclass, replace  
  
@dataclass(frozen=True)  
class User: name: str; age: int  
  
user = User("Alice", 30)  
older = replace(user, age=31)
```

JS

```
"use strict";  
  
const user = Object.freeze({ name: "Alice", age: 30 });  
const older = { ...user, age: 31 };
```



Иннополис

17-18 апреля

ИММУТАБЕЛЬНОСТЬ

Задача: обновить одно поле записи без изменения оригинала

OCaml

```
type user = { name: string; age: int }  
  
let user = { name = "Alice"; age = 30 }  
let older = { user with age = 31 }
```

Scala

```
case class User(name: String, age: Int)  
  
val user = User("Alice", 30)  
val older = user.copy(age = 31)
```

Rust

```
struct User { name: String, age: u32 }  
  
let user = User { name: "Alice".into(), age: 30 };  
let older = User { age: 31, ..user };  
  
let mut draft = User { name: "Bob".into(), age: 25 };  
draft.age = 26;
```

Python

```
from dataclasses import dataclass, replace  
  
@dataclass(frozen=True)  
class User: name: str; age: int  
  
user = User("Alice", 30)  
older = replace(user, age=31)
```

JS

```
"use strict";  
  
const user = Object.freeze({ name: "Alice", age: 30 });  
const older = { ...user, age: 31 };
```



Иннополис

17-18 апреля

ИММУТАБЕЛЬНОСТЬ

Задача: обновить одно поле записи без изменения оригинала

OCaml

```
type user = { name: string; age: int }  
  
let user = { name = "Alice"; age = 30 }  
let older = { user with age = 31 }
```

Scala

```
case class User(name: String, age: Int)  
  
val user = User("Alice", 30)  
val older = user.copy(age = 31)
```

Rust

```
struct User { name: String, age: u32 }  
  
let user = User { name: "Alice".into(), age: 30 };  
let older = User { age: 31, ..user };  
  
let mut draft = User { name: "Bob".into(), age: 25 };  
draft.age = 26;
```

Python

```
from dataclasses import dataclass, replace  
  
@dataclass(frozen=True)  
class User: name: str; age: int  
  
user = User("Alice", 30)  
older = replace(user, age=31)
```

JS

```
"use strict";  
  
const user = Object.freeze({ name: "Alice", age: 30 });  
const older = { ...user, age: 31 };
```



Иннополис

17-18 апреля

ИММУТАБЕЛЬНОСТЬ

Задача: обновить одно поле записи без изменения оригинала

OCaml

```
type user = { name: string; age: int }  
  
let user = { name = "Alice"; age = 30 }  
let older = { user with age = 31 }
```

Scala

```
case class User(name: String, age: Int)  
  
val user = User("Alice", 30)  
val older = user.copy(age = 31)
```

Rust

```
struct User { name: String, age: u32 }  
  
let user = User { name: "Alice".into(), age: 30 };  
let older = User { age: 31, ..user };  
  
let mut draft = User { name: "Bob".into(), age: 25 };  
draft.age = 26;
```

Python

```
from dataclasses import dataclass, replace  
  
@dataclass(frozen=True)  
class User: name: str; age: int  
  
user = User("Alice", 30)  
older = replace(user, age=31)
```

JS

```
"use strict";  
  
const user = Object.freeze({ name: "Alice", age: 30 });  
const older = { ...user, age: 31 };
```


Итог: пять свойств × пять языков

	OCaml	Scala	Rust	Python	JS
Декларативность	+++	+++	+++	++	++
Выражения	+++	++	+++	+	+
Функции как значения	+++	+++	++	++	+++
Чистота функций	+++	++	++	+	+
Иммутабельность	+++	++	+++	+	+

+++ — в дизайне языка, идиоматично ++ — поддерживается, требует дисциплины + — возможно, но нетипично



Иннополис

17-18 апреля

ЗАДАНИЕ НА ДОМ

1. Какие из пяти свойств присутствуют в вашем **текущем проекте**?
2. Какие из них язык гарантирует **by design**, а какие — **конвенцией**?
3. Где явная иммутабельность или декларативный стиль снизили бы **когнитивную нагрузку** в последнем PR?



Иннополис

17-18 апреля

БОНУС-ТРЕК: ФП В PRODUCTION



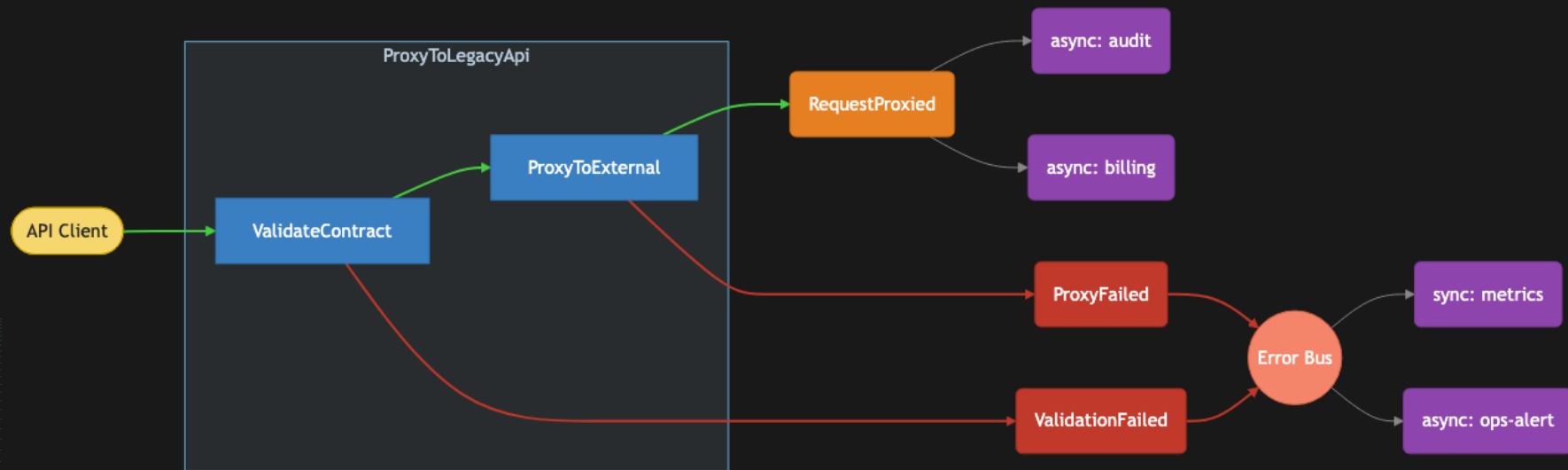
Иннополис

17-18 апреля

PRODUCTION: RUBY-СТЕК

- Фреймворк: **Rails**
- **dry-types** — Type constructors, Value Objects
- **dry-validation** — Contract DSL
- **dry-monads** — Result, До-нотация
- **dry-matcher** — Pattern matching на Result
- Архитектура: **CQRS + DDD, EventStore**

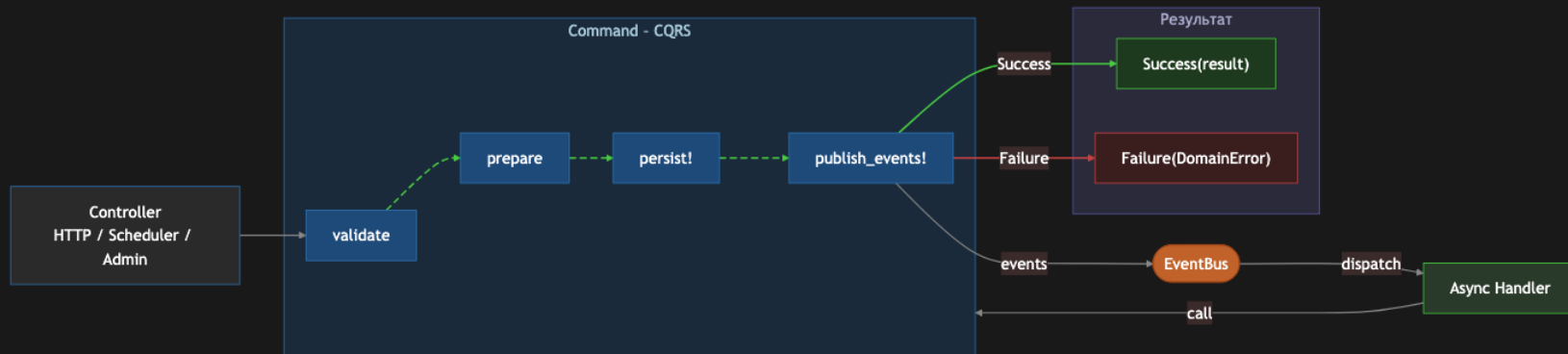
EVENT FLOW: ТИПОВОЙ СЦЕНАРИЙ



ФП → DDD: МАППИНГ

Принципы ФП	DDD implementation
Функции как значения	Command: функциональный объект
Чистота функций	Command: изолированная операция БЛ
Иммутабельность	иммутабельные DTO
Декларативность	монадный DSL
Выражения	ROP → Result

АРХИТЕКТУРА КОМАНДЫ





Иннополис

17-18 апреля

ROP В КОМАНДЕ

Railway-Oriented Programming через **.bind**

Ruby

```
def call(raw_input = {})
  @context_result
    .bind { do_validate(raw_input) }
    .bind { |input| do_prepare(**input) }
    .bind { |input| do_persist!(**input) }
    .bind { |result| publish_events!(result) }
end
```




Иннополис

17-18 апреля

DO-НОТАЦИЯ

yield разворачивает Result внутри монады

Ruby

```
include Dry::Monads::Do.for(:load_entity)
```

```
def load_entity(id:)
  owner  = yield find_owner(id)
  config = yield load_config(owner)
  state  = yield resolve_state(config)
```

```
  Success(owner:, config:, state:)
```

```
end
```



Иннополис

17-18 апреля

ТИПОВЫЕ КОНТРАКТЫ

dry-types: конструкторы типов как значения

Ruby

```
module DomainTypes
```

```
  DownCasedString = Types::String.constructor { |v| v&.downcase }
```

```
  Email = DownCasedString.constrained(format: EMAIL_REGEXP)
```

```
  Uuid = StrippedString.constrained(format: UUID_REGEXP)
```

```
module HttpTypes
```

```
  Method = Types::String.enum(*%w[GET POST PUT PATCH DELETE])
```

```
  SafeMethod = Types::String.enum(*%w[GET HEAD OPTIONS])
```

```
end
```

```
end
```



Иннополис

17-18 апреля

ДОМЕННЫЕ ОШИБКИ КАК ЗНАЧЕНИЯ

DomainError — **иммутабельный** value object

Ruby

```
class DomainError
  extend Dry::Initializer

  option :field, optional: true
  option :meta,  optional: true

  def to_monad = Dry::Monads::Failure(self)
end

ExternalApiError = Class.new(DomainError) do
  option :path
  option :error_type # :timeout | :connection_failed
  option :http_status # 502 | 504
end
```



Иннополис

17-18 апреля

БИЗНЕС-DSL

DomainFailureMatcher: декларативный **pattern matching**

Ruby

```
MATCHER = DomainFailureMatcher.define_matcher
```

```
MATCHER.call(command.call(params)) do |m|  
  m.success          { |result| render_ok(result) }  
  m.failure(ExternalApiError) { |e| render_proxy_error(e) }  
  m.failure(ContractError)   { |e| render_validation(e) }  
  m.failure           { |e| render_unexpected(e) }  
end
```



Иннополис

17-18 апреля

СИНТЕЗ: ПЯТЬ ПРИНЦИПОВ В PRODUCTION

- **Декларативность** → Contract **DSL**, **Result** pipeline
- **Выражения** → Каждый **.bind** возвращает **Result**
- **Функции как значения** → **.bind**, лямбды в **DSL**
- **Чистота функций** → **Command** без глобального состояния
- **Иммутабельность** → **Type constructors**, **DomainError** как value object



Иннополис

17-18 апреля

Павел Аргентов

Tg: @argent_smith



2026